Introduction

The primary purpose of these exercises was to learn how to create ASP.NET Core Web Applications utilizing both Razor Pages and the Model-View-Controller (MVC) pattern. Throughout the lab, I applied the Code First approach, which allowed me to define my database schema directly using C# classes, while relying on Entity Framework Core to automatically generate the backend database.

Exercises

Exercise 1

In this exercise, I successfully set up a new ASP.NET Core Web Application utilizing Razor Pages.

- I began by creating the project and installing the necessary Entity Framework Core NuGet packages, specifically Microsoft.EntityFrameworkCore.SqlServer and Microsoft.EntityFrameworkCore.Tools.
- I defined the data schema by creating several C# entity classes, including Department, Course, Instructor, Student, and Enrollment.
- To manage the database connection, I created a SchoolContext class inheriting from DbContext and configured the SQL Server connection string within the appsettings.json file.
- Using the Package Manager Console, I executed the Add-Migration InitialCreate and Update-Database commands to generate the physical database based on my models.
- Finally, I used the scaffolding engine to automatically generate Razor Pages equipped with full Create, Read, Update, and Delete (CRUD) operations for the data models, and subsequently modified the application's navigation menu to link to these new pages.

SchoolAppCoreRazor Home Privacy Students Courses Instructors Enrollments Departments

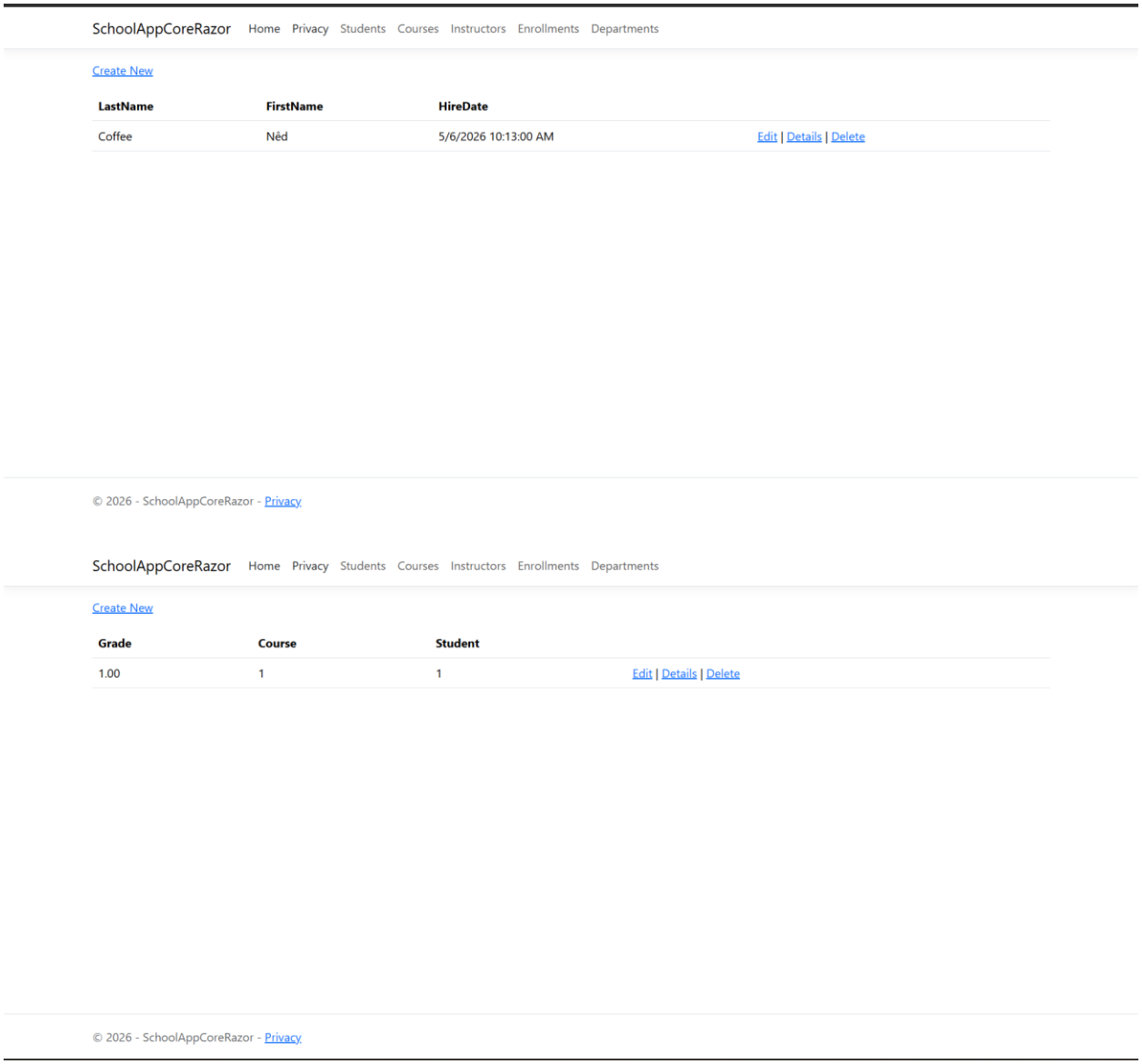
Create New

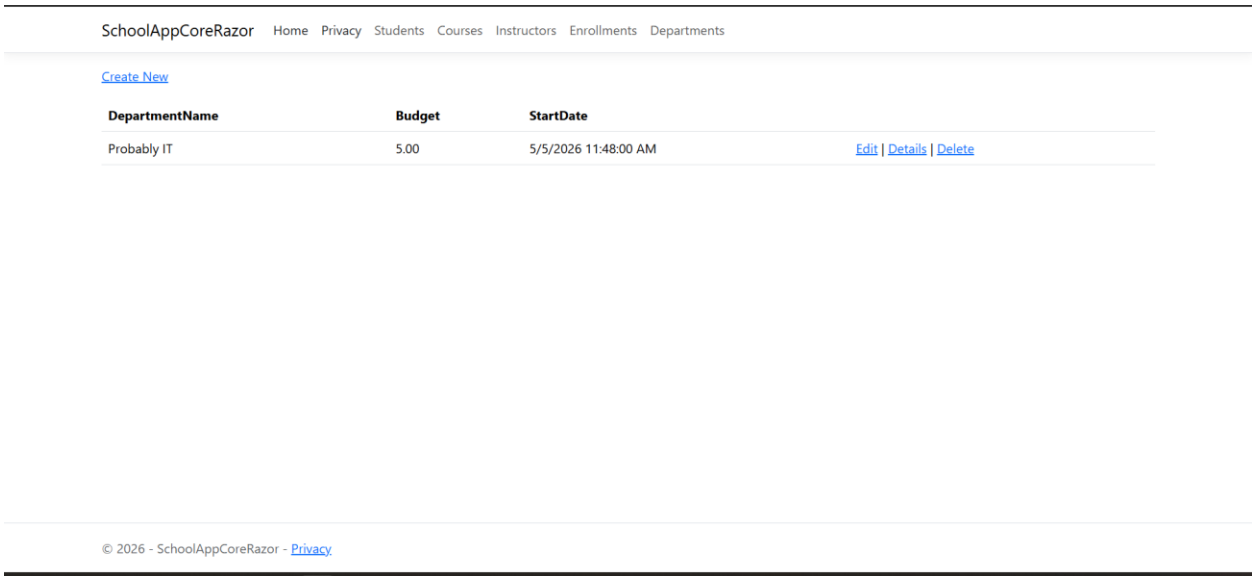
LastName	FirstName	EnrollmentDate	
Smith	John	5/1/2026 11:49:00 AM	Edit | Details | Delete

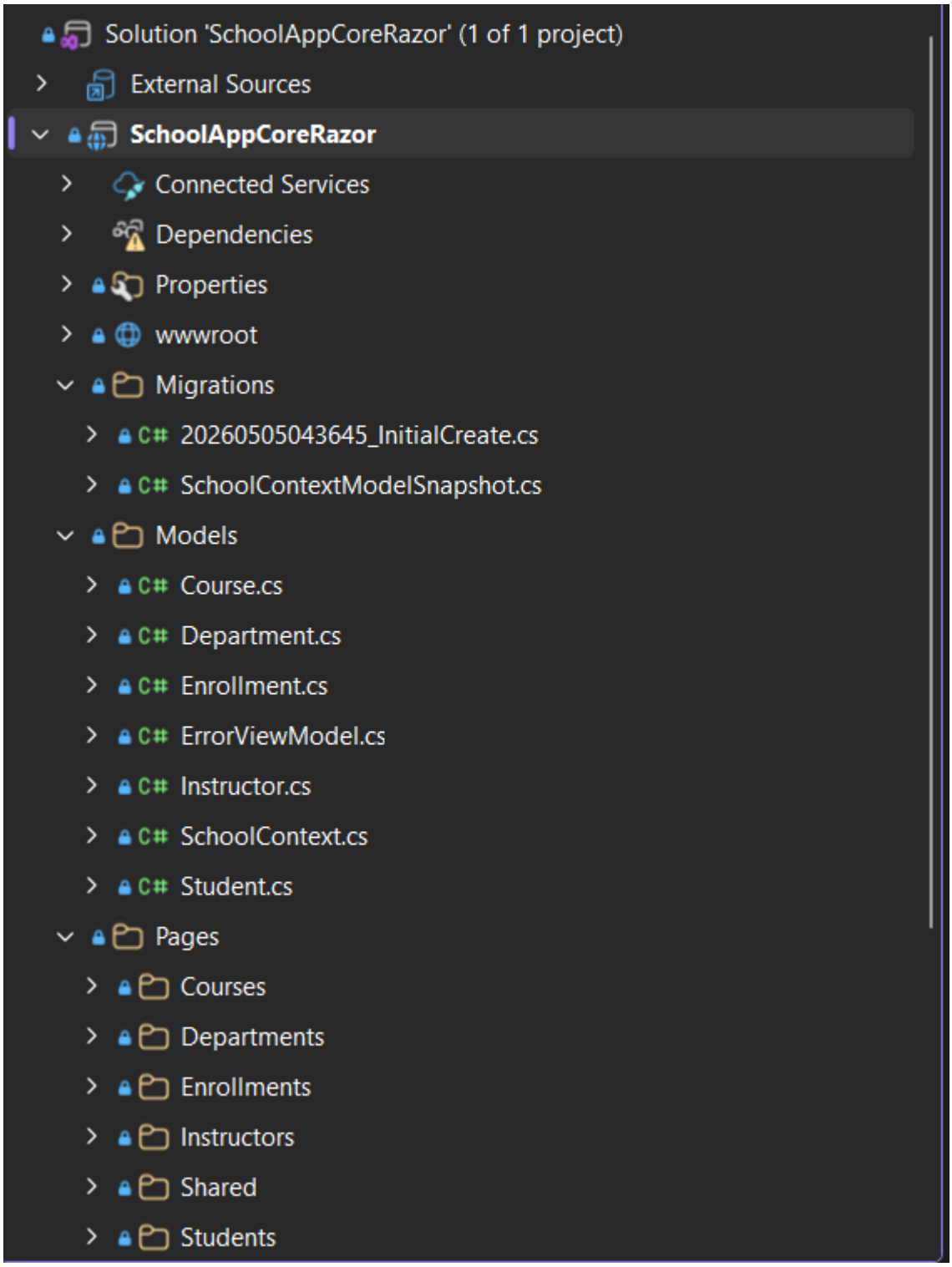
SchoolAppCoreRazor Home Privacy Students Courses Instructors Enrollments Departments

Create New

Title	Credits	Department	
Oh I got this wrong	5	1	Edit | Details | Delete
Karate	4	1	Edit | Details | Delete


SchoolAppCoreRazor
Home
Privacy
Students
Courses
Instructors
Enrollments
Departments
Create New
LastName
FirstName
HireDate
Coffee
Nêd
5/6/2026 10:13:00 AM
Edit | Details | Delete
© 2026 - SchoolAppCoreRazor - Privacy
SchoolAppCoreRazor
Home
Privacy
Students
Courses
Instructors
Enrollments
Departments
Create New
Grade
Course
Student
1.00
1
1
Edit | Details | Delete
© 2026 - SchoolAppCoreRazor - Privacy

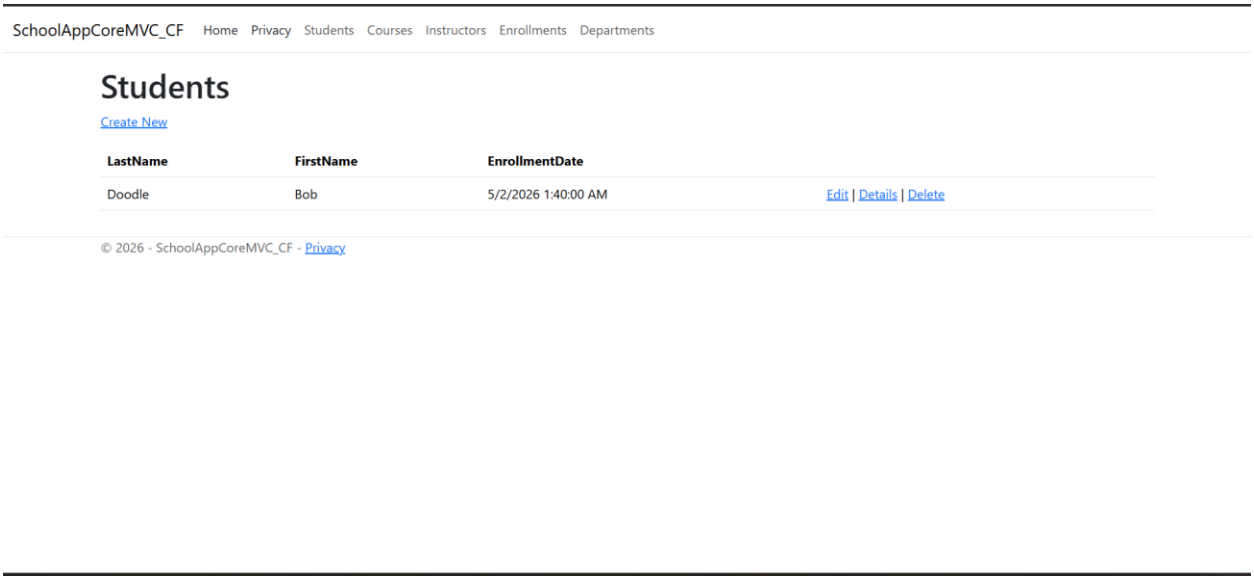

SchoolAppCoreRazor
Home
Privacy
Students
Courses
Instructors
Enrollments
Departments
Create New
DepartmentName
Budget
StartDate
Probably IT
5.00
5/5/2026 11:48:00 AM
Edit | Details | Delete
© 2026 - SchoolAppCoreRazor - Privacy

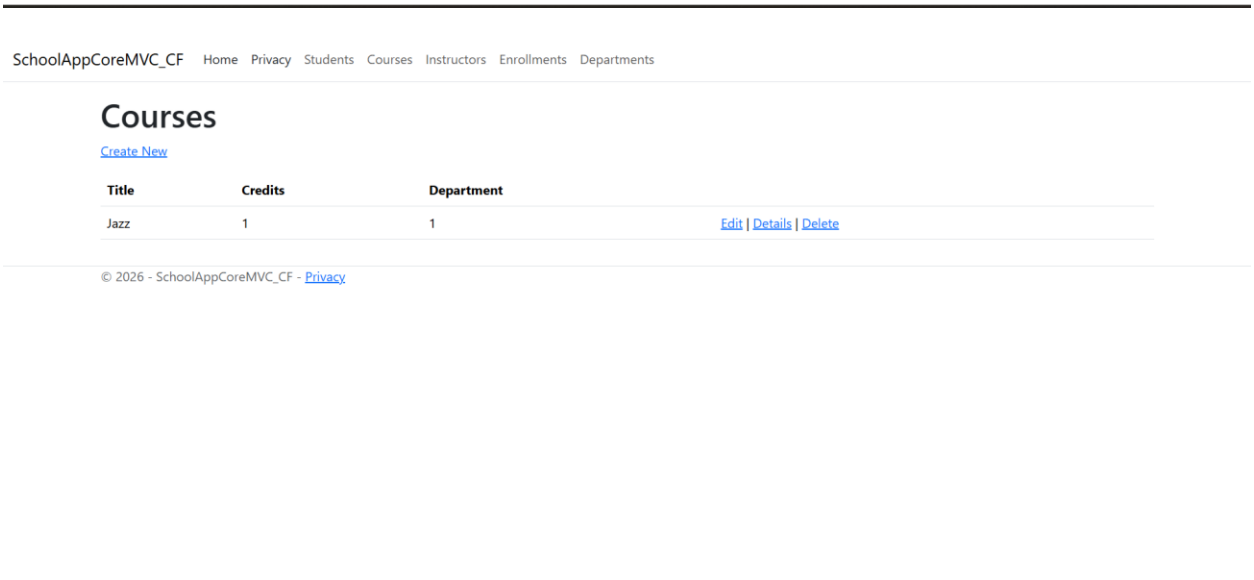


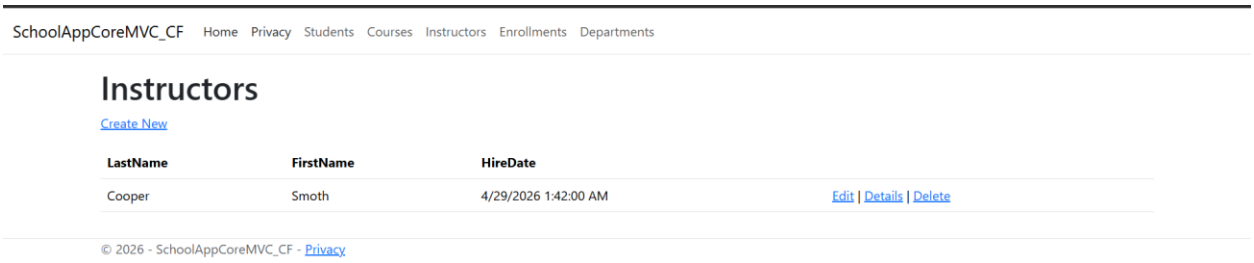
Exercise 2

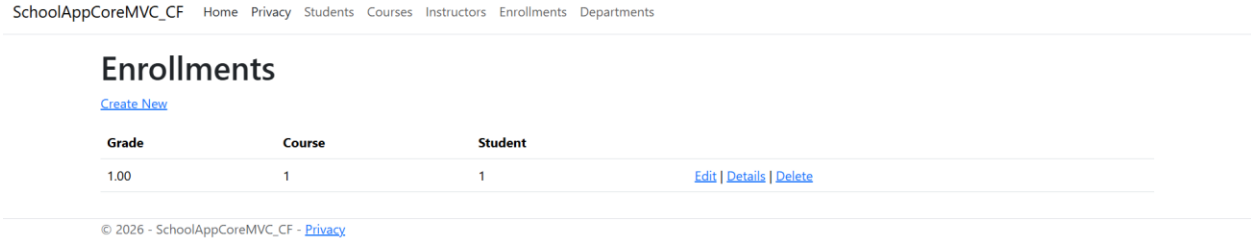
This exercise focused on building a comparable application, but this time leveraging the MVC architectural pattern.

- I created a new project using the Web Application (Model-View-Controller) template and installed the requisite Entity Framework Core packages.
- I reused the entity classes from the previous exercise, placing them in the Models folder, and set up the SchoolContext and database connection string similarly to Exercise 1.
- After running the database migrations, I used the scaffolding tool to generate MVC Controllers with views using Entity Framework.
- This exercise practically demonstrated the structural differences between the frameworks; specifically, how MVC enforces a clear separation of concerns among Controllers, Views, and Models, in contrast to the page-centric nature of Razor Pages.





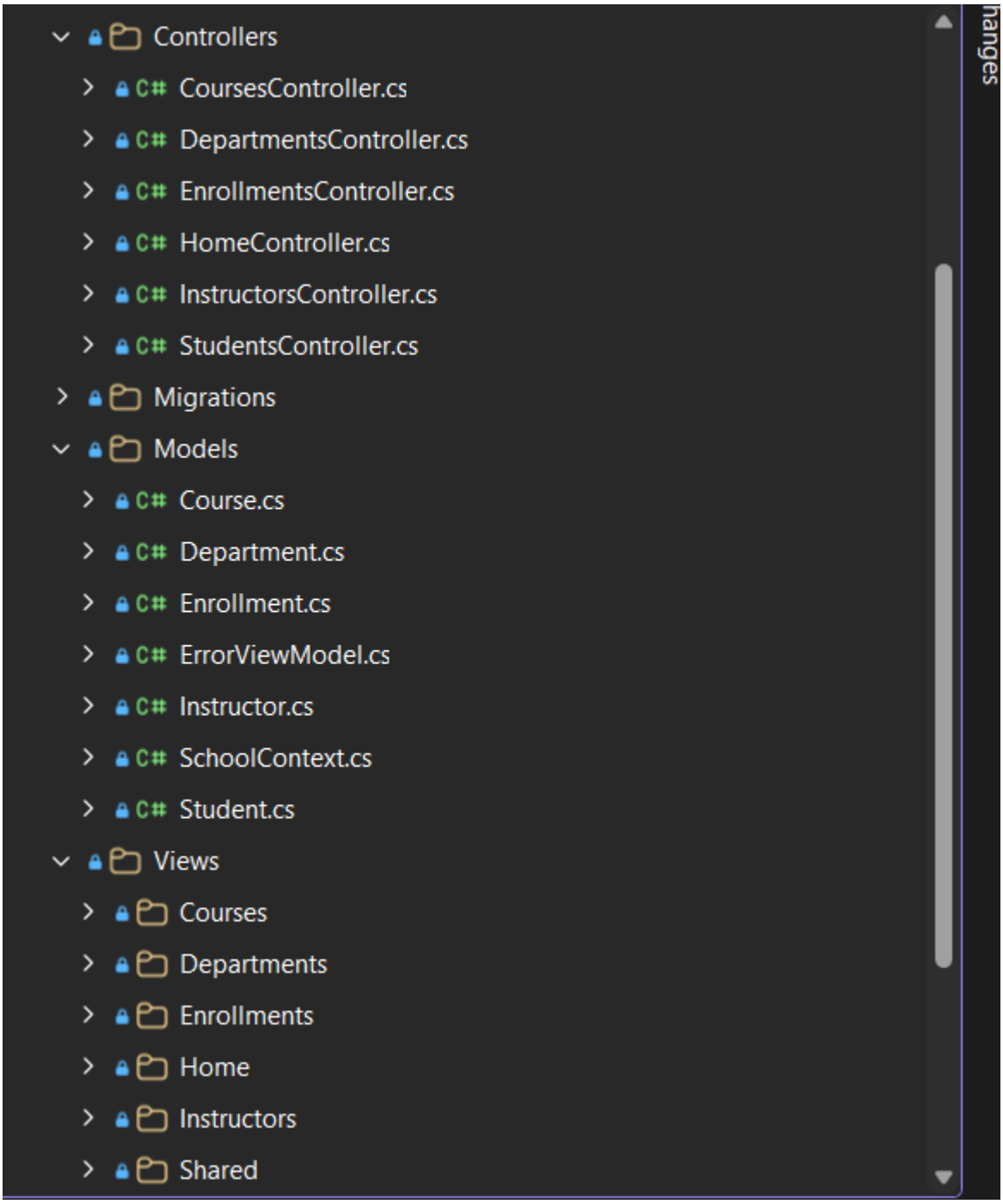

SchoolAppCoreMVC_CF
Home
Privacy
Students
Courses
Instructors
Enrollments
Departments
Instructors
Create New
LastName
FirstName
HireDate
Cooper
Smoth
4/29/2026 1:42:00 AM
Edit | Details | Delete
© 2026 - SchoolAppCoreMVC_CF - Privacy


SchoolAppCoreMVC_CF
Home
Privacy
Students
Courses
Instructors
Enrollments
Departments
Enrollments
Create New
Grade
Course
Student
1.00
1
1
Edit | Details | Delete
© 2026 - SchoolAppCoreMVC_CF - Privacy

Departments

Create New

DepartmentName	Budget	StartDate	
Music	2.00	5/7/2026 1:41:00 AM	Edit | Details | Delete

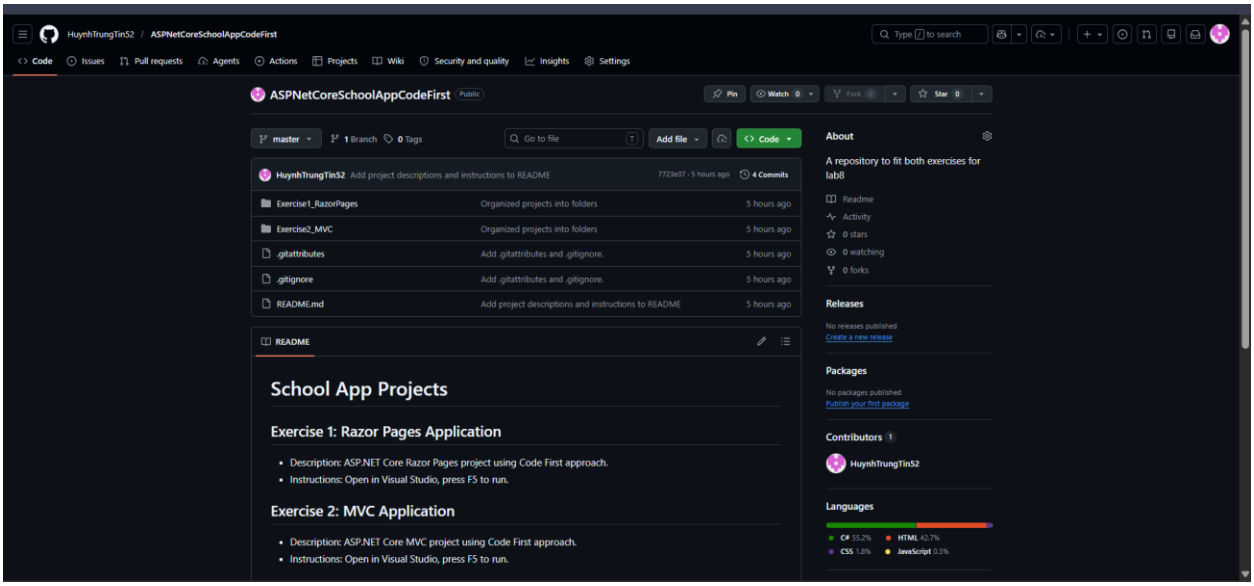


Exercise 3

The final phase involved using Git source control to upload and structure my work.

- Using Visual Studio's Git integration, I staged and committed my initial changes for both the Razor Pages and MVC projects.
- I pushed the commits to a remote GitHub repository.

- To keep the repository clean, I organized the repository by creating specific folders named Exercise1_RazorPages and Exercise2_MVC, and moved the respective projects into them.
- Lastly, I added a README.md file detailing the descriptions and run instructions for both web applications.



Conclusion

Overall, these exercises provided me with valuable hands-on experience in building web applications with ASP.NET Core and managing databases via the Entity Framework Core Code-First approach. Developing the same core application twice highlighted the distinct advantages of each framework: Razor Pages provided a streamlined, rapid development experience for simple page-based navigation, whereas MVC offered robust flexibility and separation of concerns better suited for complex interactions. The primary challenge I faced was ensuring that all connection strings, namespaces, and scaffolding configurations were correctly mapped during the transitions, but testing the final applications confirmed that the web forms successfully interacted with the database.